Receiver-Owned Bilateral Admission for Cross-Organization Agent Tool Calls

Anonymous Author(s)

ABSTRACT

A receiver cannot authorize a cross-organization agent call from a vendor signature alone. The signature authenticates bytes, but it does not show that those bytes name the receiver's expected treaty, request, outcome, continuation, or policy. We present receiver-owned bilateral admission, a protocol that performs that check before the receiving kernel invokes a tool. The receiver resolves treaty state and authorized keys from its own stores, verifies two distinct configured keys over one canonical DSSE predicate, and checks that every referenced request, outcome, continuation, lineage statement, and receipt belongs to the same invocation. The Rust implementation rejects missing, stale, replayed, and mismatched evidence before dispatch. A Lean 4 model gives executable semantics to the bounded receipt predicates and proves that its finite-domain refinement checker accepts exactly when the candidate predicate admits no receipt in the supplied domain that the current predicate rejects. An independently written Rust reference evaluator matched the runtime evaluator on every atom and on 1,024 generated cases for each of three compound properties. On a 12-core Arm Neoverse-N1 workstation, the pre-dispatch admission path for an admitted cross-organization call, including receiver store resolution, treaty binding checks, envelope verification, dispatch to a counting tool server, and the signed SQLite receipt, measured 13.863 ms p50 and 26.176 ms p99 over 200 invocations. A unanimous policy denial, which returns at the policy-summary check and writes a signed SQLite denial receipt, measured 7.073 ms p50 and 18.646 ms p99 over 400 invocations with zero tool invocations. A separate three-vendor local workflow that produced and verified the complete buyer package measured 2.089 s p50 and 2.142 s p99 over 100 runs. All 20 named negative cases passed.

1 INTRODUCTION

Consider a buyer agent that asks a vendor agent to stage a refund. The vendor returns a correctly signed receipt. The buyer still has to reject it if the receipt belongs to another treaty, binds different arguments, follows an expired continuation, omits required lineage, or records a policy result the buyer does not accept. A signature identifies a key; it does not decide whether the receiver should admit the action. An audit performed after execution is too late if the tool has already changed the vendor ledger.

Receiver-owned admission moves that decision into the receiving kernel. For a cross-organization request, the kernel resolves treaty records, authorized keys, continuations, and prior receipts from receiver-controlled stores. It checks that the evidence describes one invocation and returns a decision before tool dispatch. Missing or inconsistent context denies, request metadata cannot install a trust root, and each continuation can be consumed only once.

The protocol combines three familiar objects in a stricter order. A receipt is a kernel-signed record of a mediated tool decision over canonical bytes. A treaty fixes the participants, keys, permitted action classes, and validity period. A bilateral predicate is a canonical DSSE statement signed by two configured keys and bound to the request, outcome, treaty, continuation, lineage, and both receipt digests [15, 16]. The receiver checks that predicate only after loading the referenced state from its own stores.

The difficult part is preventing substitution. A sender must not be able to replace the treaty while retaining a valid receipt signature, exchange one set of request arguments for another, replay a continuation, or combine signatures over different statements. Chio therefore verifies the complete evidence graph at the dispatch boundary. Two distinct keys over the same bytes are necessary, but they do not by themselves prove independent organizational control. Key custody remains a deployment assumption.

We implement this path in Chio, a capability-mediated runtime for agent tool calls. The kernel performs its usual capability checks, then runs explicit treaty, evidence, continuation, and DSSE validation for cross-organization requests. Accepted treaty material is carried into receipt construction as a validated type, which prevents the signed receipt from being assembled from a different request or participant snapshot. A separate bounded predicate library supports executable modeling and differential testing; it is not an alternate authorization path.

This paper makes four contributions:

- **A bilateral admission protocol.** One canonical DSSE predicate binds the request, outcome, treaty, continuation, lineage, leases, governance references, and both receipt digests to two distinct configured signer keys.
- **A receiver-owned pre-dispatch implementation.** The kernel resolves trust material from local stores, rejects request-supplied trust, consumes continuations once, and denies stale, replayed, malformed, or inconsistent evidence before invoking the tool.
- **A small executable model.** Lean 4 defines the bounded receipt predicate language and proves the exact condition checked by finite-domain refinement. An independently written Rust model is tested against the runtime evaluator on every atom and on generated compound cases.
- **An adversarial and performance evaluation.** A dispatch-capable benchmark measures the pre-dispatch admission path for admitted and denied calls and asserts the tool invocation count. A three-vendor SQLite workflow produces and verifies a buyer package, while 20 named negative cases exercise binding, freshness, trust, syntax, and policy failures.

The receiver can prevent a foreign agent action from crossing its tool boundary when the supplied evidence does not match receiver-controlled state. The protocol makes no claim that either signer is honest or that two configured keys imply two independent operators.

2 SECURITY CONTEXT AND THREAT MODEL

Capability mediation. Capability systems replace ambient authority with explicit, attenuable references. KeyKOS and EROS made this discipline an operating-system primitive. Capsicum brought capability mode to UNIX, and seL4 connected a microkernel authority model to machine-checked functional correctness [7, 9, 17, 22]. Object-capability work explains authority as reachability and attenuation rather than membership in a global access-control list [12, 13]. Chio applies that discipline to agent tool calls: the untrusted agent supplies a signed capability, while a trusted kernel validates it and mediates the tool server.

Agent isolation and cross-domain authority. IsolateGPT confines agent applications behind information-flow gates; SAGA distributes user-controlled authorization tokens; recent agent-security work identifies confused-deputy and authority-boundary failures as a central systems problem [2, 20, 23]. These systems primarily protect a local principal. Bilateral admission begins when the evidence crosses an administrative boundary and the receiver must decide whether a foreign action belongs to its own authorized history.

Signed statements are necessary but incomplete. DSSE authenticates a payload and predicate type, while in-toto and SLSA bind subjects to supply-chain provenance [16, 19, 21]. A valid provenance statement does not establish receiver authorization for a tool call. The receiver additionally needs freshness, request and outcome binding, participant and treaty binding, continuation state, and its own policy verdict. Chio retains DSSE's canonical signed envelope but defines an action-admission predicate rather than a build-provenance predicate.

Trusted components. We trust the receiving kernel, its cryptographic and canonical-JSON libraries, and its locally provisioned stores. The stores contain expected peer keys, treaty scope, ladder manifests, leases, governance receipts, revocation state, continuations, lineage, and resolved receipts. We assume Ed25519 and SHA-256 behave as specified and that the receiver classifies federated origin correctly. A compromised receiver can authorize anything and lies outside the property proved here.

Adversary. The adversary controls the agent request and may present arbitrary metadata and syntactically valid signed objects. It may replay old material; mix artifacts from different invocations; alter the treaty, request, outcome, receipt, lineage, continuation, or predicate type; omit a signature or governance record; use stale leases; duplicate a signer key identifier; or submit a correctly signed unanimous denial. It may control a remote vendor kernel and any request-carried trust data. The receiver must deny these cases before its tool runs.

The adversary may also control two private keys that the receiver has separately authorized. Cryptography cannot distinguish this condition from two administrative principals. We therefore test rejection of an identical repeated key but record distinct-controller independence as assumption PS-A-01. TEE-backed key custody or external organizational governance could strengthen that assumption, but neither is part of the evaluated construction.

Security objective. Let q be a federated tool request and $A_R(q)$ the receiving kernel's admission decision. The implementation objective is:

$$A_R(q) = \text{allow} \implies \text{coreAllows}(q) \wedge \text{receiverEvidenceAccepts}(q).$$

The second predicate means that receiver-resolved treaty evidence is fresh, mutually bound, strictly verified, and accepted by receiver policy. If any premise fails, the receiver returns a denial before invocation. This is a local safety property. It neither forces the sender to record the same history nor establishes the truth of claims made by a compromised but authorized signer.

3 RECEIVER-OWNED BILATERAL ADMISSION

The protocol separates evidence produced by the sender from authority retained by the receiver. A sender may supply receipts and a DSSE envelope. It may not supply the receiver's expected keys, treaty record, trust bundle, or continuation state. Those objects are resolved by identifier from receiver-owned stores.

3.1 Artifacts and bindings

Receipt. A Chio receipt records the capability identifier, tool server and name, canonical action hash, allow or deny decision, policy hash, evidence, metadata, trust level, and kernel public key. The body is signed over canonical JSON. A valid vendor receipt proves only that the corresponding key signed that body. The receiver still has to decide whether the receipt belongs to the current action.

Treaty scope and ladder. A *TreatyScope* fixes a treaty identifier, participant kernel identifiers and public keys, hashes of participant ladder manifests, allowed action classes, validity interval, revocation epoch, and trust-bundle digest. Each ladder manifest assigns an enforcement mode and evidence requirement to an action class. The finite order is observation, guarded, receipt-backed, partition-contingency, and maintenance. Intersection takes the stricter participant mode and rejects a destructive class below receipt-backed. An unknown class denies. Each action class also declares a consistency model, one of crdt-commutative, totally-ordered, or quorum-required; quorum-required is a consistency model, not a ladder mode.

Continuation and lineage. A *CrossKernelContinuation* links child work to a parent receipt and session anchor, capability, action class, target tool, nonce, source and target kernels, and validity interval. A *ReceiptLineageStatement* binds the parent and child receipt digests to that continuation and to the bilateral invocation. The receiver consumes the continuation during admission evaluation, records the consumed identifier in the accepted result, and releases it only when pre-dispatch execution denies or aborts.

Bilateral predicate. The strict predicate type is `chio.bilateral-cosign-invocation.v1`. Its treaty reference binds the treaty scope and ladder-intersection digests; admission report; continuation; lineage bundle; action class and consistency model; request and outcome digests; local and remote receipt digests; lease and governance references; and participant kernel identifiers. DSSE signs the canonical statement through its pre-authentication encoding (PAE) [16]. The envelope contains exactly two signatures whose

key identifiers must be distinct and match the configured public keys. Figure 1 shows the artifact.

Two details matter. Both signatures must cover the same predicate bytes; two signatures over different statements do not compose into bilateral evidence. Distinct keys also do not establish distinct controllers. The verifier checks the bytes and key identifiers, while deployment policy is responsible for key custody.

3.2 Admission sequence

Figure 2 shows the receiver's control flow.

(1) The kernel first performs its ordinary capability, scope, revocation, and guard checks. Budget admission runs after the treaty hook, so a treaty denial never charges the budget.
(2) Trusted runtime state classifies the request. Local requests follow the ordinary local admission path. A request classified as federated without complete treaty context denies.
(3) The hook parses only identifiers and bindings from request context. Any attempt to carry a trust root, peer directory, signing key, or dynamic trust bundle in request metadata denies.
(4) Receiver stores resolve the treaty scope, ladder intersection, continuation, lineage bundle, bilateral invocation, bilateral DSSE envelope, and the admission bundle that names the capability lease and governance receipt.
(5) Validators check schema, canonical hashes, freshness, participant sets, action class, consistency model, required evidence, continuation origin and target, and cross-artifact equality.
(6) The hook checks the policy summary, every treaty binding, the lease and governance references against its admission bundle, and the signer set against the treaty participants. The strict envelope verifier then checks the payload type, exactly two signatures, canonical payload bytes, statement and predicate types, one subject, two distinct key identifiers, and both signatures. The kernel separately requires the federated origin to be a pinned, fresh federation peer before the hook runs, while the key-fingerprint comparison against the pinned peer set, ladder-manifest freshness, and the revocation epoch are checked by the offline buyer verifier (Section 5).
(7) Admission either returns a named failure before tool invocation or permits dispatch. On the admitted path, the runtime emits bound receipt metadata and assembles buyer-review evidence.

Receiver-owned policy. Explicit Rust validators and the strict bilateral verifier make the hook's admission decision. The bounded evaluator in Section 4 operates on a receipt view constructed after those checks and is not a substitute authorization path.

Failure evidence. The negative matrix assigns each attack a stable identifier, enforcement phase, and expected diagnostic. Kernel tests and the dispatch-capable benchmark also inspect the tool invocation counter. Envelope-only cases stop earlier, before entering an API that can dispatch a tool.

4 BOUNDED PREDICATE MODEL

The Lean development specifies the small predicate library used for differential testing. Its input is a receipt view that has already been built from validated artifacts:

$$r = (id, hash, action, participants, \\ modeRank, liveContinuations, decision, \\ failureCode, evidenceDigests).$$

The model starts at this data boundary. Signature verification, canonical serialization, clocks, durable storage, and construction of r are implementation obligations described in Section 5.

4.1 Predicate Semantics

The predicate syntax is

$$p ::= \top \mid \bot \mid \mathsf{atom}(a) \mid p \wedge p \mid p \vee p \mid \neg p.$$

The nine atom forms test receipt identity, participant membership, action class, minimum ladder rank, receipt hash, live continuation membership, decision, failure code, or an evidence-class and digest pair. The recursive function $\mathsf{evaluate}(p, r)$ returns a Boolean for every predicate and receipt view.

A constitution is a finite list of predicates:

$$\mathsf{admits}(K, r) = \bigwedge_{p \in K} \mathsf{evaluate}(p, r).$$

The language covers the receipt checks exercised by the reference and runtime evaluators while remaining small enough to execute independently in Lean and Rust.

4.2 Finite-Domain Refinement

For a candidate constitution K', a current constitution K, and a supplied list of receipt views D, the checker computes

$$\mathsf{refinesOn}(K', K, D) = \bigwedge_{r \in D} \left(\mathsf{admits}(K', r) \Rightarrow \mathsf{admits}(K, r)\right).$$

The direction is important: on D, the candidate may reject more receipts, but it may not admit a receipt rejected by the current constitution.

The theorem `finite_refinement_exact` states

$$\mathsf{refinesOn}(K', K, D) = \mathsf{true} \\ \iff \forall r \in D.\ \mathsf{admits}(K', r) = \mathsf{true} \\ \Rightarrow \mathsf{admits}(K, r) = \mathsf{true}.$$

Its forward direction, `finite_refinement_sound`, is the condition a caller can rely on after a successful check. Both theorems are checked by Lean 4 without project-specific axioms.

This guarantee is limited to D. An empty list passes vacuously, and a list that omits a relevant receipt says nothing about that receipt. The model proves the checker correct for its input; it does not prove that a caller supplied a complete deployment domain.

4.3 Relation to Rust

The Rust reference interpreter in `formal/diff-tests` was written separately from `chio-runtime-core`. Both implement the receipt fields, atom forms, Boolean syntax, constitution admission, and finite-domain check shown above. Tests enumerate every atom and generate 1,024 cases for each of compound predicate evaluation, constitution admission, and finite-domain refinement.

Those tests compare the two Rust implementations. Lean checks the mathematical definition and the refinement theorem, but the

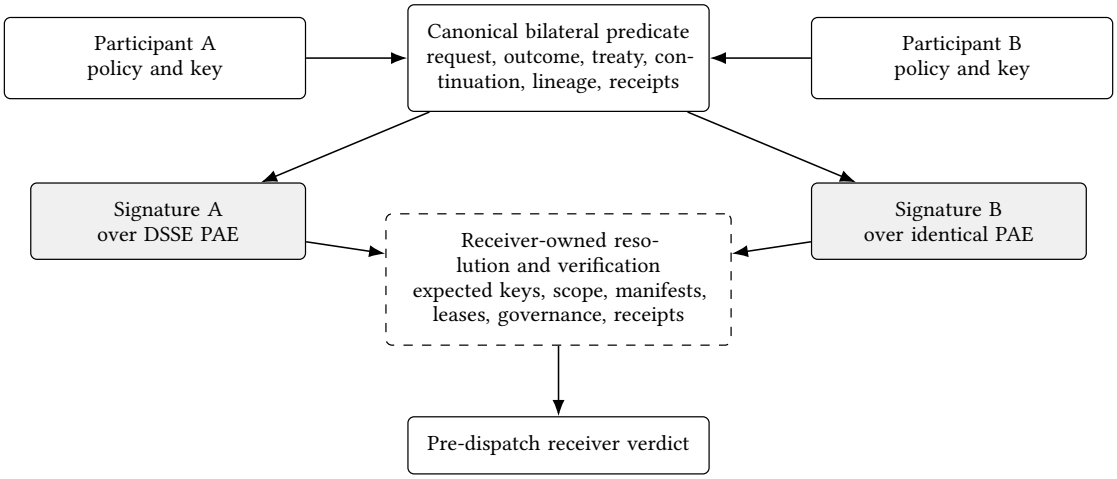

Figure 1: The bilateral artifact. Participant policy produces a common treaty-bound predicate; two configured keys sign the same DSSE PAE bytes. The receiver then resolves and checks every referenced object from its own trust context.

Rust evaluator is neither generated from Lean nor connected to it by a refinement proof. Rust also rejects predicates that exceed its size, depth, or node limits before applying these semantics; those limits are not modeled in Lean. The receiver admission hook performs many checks outside this bounded library. The paper evaluates those checks through focused runtime tests and the dispatch-capable negative path rather than treating the Lean model as a proof of the whole receiver.

5 IMPLEMENTATION

The implementation follows the receiver's control flow. The kernel first validates the capability, scope, revocation state, and guards. A cross-organization request then enters `ChioRuntimeAdmissionHook::evaluate`, which loads the referenced artifacts, checks the treaty and continuation, verifies the bilateral statement, and returns either a named denial or an accepted result. Budget admission runs after the hook returns, so a treaty denial never charges the capability budget. The tool server is invoked only after these stages succeed.

5.1 Receiver-Owned Resolution

Trusted runtime state classifies each request: the receiving edge sets `federated_origin_kernel_id`, and the caller cannot. Local requests follow the local admission path. A request classified as federated must carry a complete set of identifiers and bindings; malformed or missing context denies. `treaty_ref_from_request` reads only identifiers and digests from the treaty context and resolves every artifact through the hook's own `RuntimeAdmissionStores`. That identifier-only parsing plus store resolution is the structural guarantee that request metadata cannot install trust material. As an explicit second layer, the parser rejects eleven trust-shaped keys (trust roots and bundles, treaty scopes, ladder manifests, signing keys, peer directories, and dynamic trust inputs) with named request-smuggling codes.

The receiver loads the treaty scope, ladder intersection, continuation, lineage bundle, bilateral invocation, bilateral DSSE envelope, and the admission bundle that names the capability lease and governance receipt. It checks schemas, hashes, validity intervals, participant uniqueness, action-class coverage, and the receiver's minimum ladder mode. The hook consumes the continuation during admission evaluation with a single store operation, records the consumed identifier in the accepted result, and releases it only on pre-dispatch denial or abort. If release fails, the kernel keeps the continuation consumed rather than freeing it and records the failure in the denial receipt.

5.2 Bilateral Statement

Three functions verify the bilateral statement before dispatch. `verify_treaty_dsse_evidence`, inside the hook, decodes the statement, requires the strict predicate type `chio.bilateral-cosign-invocation.v1`, validates the policy-evaluation summary and requires an allow verdict, compares the treaty binding reference (treaty identifier, treaty-scope and ladder-intersection digests, continuation digest, action class, consistency model, and request digest) with the artifacts it resolved, checks the tool-argument hash against that request digest, matches the lease and governance references against its admission bundle, and requires the signer set to equal the two treaty participants with distinct public keys. It then calls `verify_chio_bilateral_dsse_envelope` and afterwards checks the lineage-bundle and bilateral-invocation bindings.

`verify_chio_bilateral_dsse_envelope` enforces the strict DSSE profile. It checks the payload type, exactly two signatures, byte equality between the payload and its canonical form, the statement and predicate types, the predicate schema, one subject carrying the canonical receipt name, two distinct configured keys with distinct key identifiers and matching fingerprints, and finally both Ed25519 signatures over the same pre-authentication encoding. Its failures carry `BilateralCoSigningError::code` values such as `dsse.malformed`, `statement.malformed`, `predicate.type_unrecognised`, and `signer.independence_required`.

After `evaluate_cross_boundary_admission` accepts, `VerifiedFederationTreatyMaterial::verify` verifies the envelope again against the participant keys, checks the participants and tool

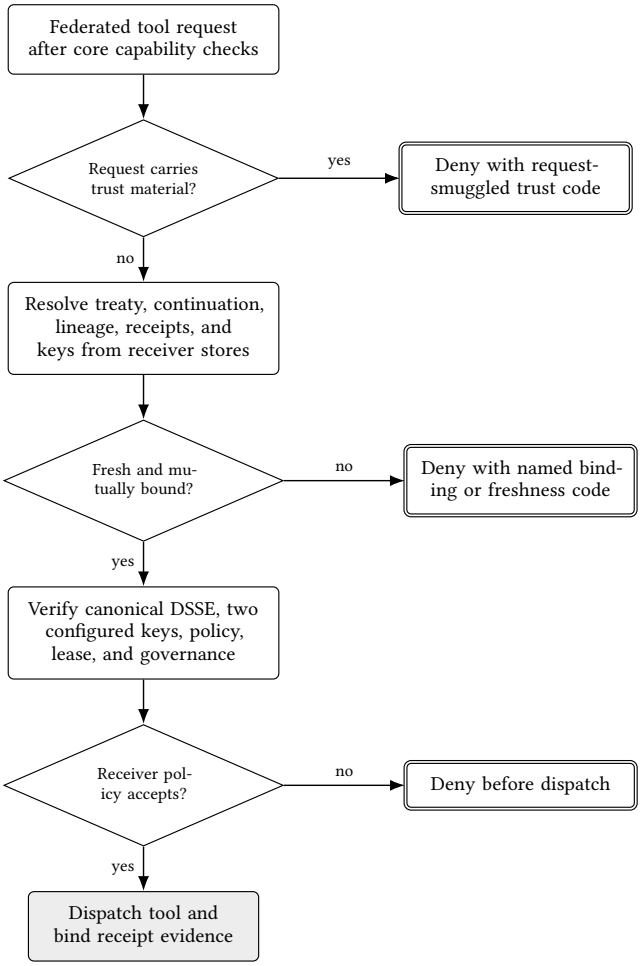

Figure 2: Receiver-side pre-dispatch admission. Trust material flows from receiver-owned stores, not from the request. Every denial terminal precedes the tool boundary.

name, recomputes the canonical request hash, requires the policy allow result, checks lease expiry against the injected clock, and compares the treaty, ladder, action, consistency, and co-sign fields with the accepted admission report. The cryptographic result is authorization by two configured keys. Whether those keys have independent organizational custody is outside the verifier.

5.3 Binding Admission to the Signed Receipt

An accepted admission result is converted into `VerifiedFederationTreatyMaterial`. Its constructor checks the request's origin and local participant, distinct participant keys, the DSSE predicate, the canonical request hash, the policy allow result, the capability lease, the treaty and ladder hashes, and the action and consistency classes. It then binds the local accepted admission-report digest into the treaty binding reference, replacing any peer-supplied value rather than comparing against it, so a locally signed receipt never carries a peer-supplied report digest. The kernel stores this object for the current request rather than retaining a loose collection of request metadata.

Receipt construction rechecks the request, participants, tool, action hash, and receipt contents against that object. Only then does it attach the treaty metadata and the resulting outcome and receipt digests to the signed receipt. This second binding prevents an accepted report from being reused to assemble a receipt for a different request or participant snapshot.

5.4 Bounded Predicate Library

The versioned document type `chio.federation.bounded-treaty-predicate.v1` rejects unknown versions, tags, and fields. It enforces a 64 KiB document limit, a 1 KiB atom string limit, and depth and node limits of 32 and 1,024. `bounded_treaty_receipt_view_from_verified_artifacts` constructs the input described in Section 4. It requires an independently authenticated admission-report digest and cross-checks the treaty, action, invocation, and continuation before returning the view.

This library exists for explicit receipt-policy evaluation and differential testing. It does not grant access on behalf of the admission hook. The hook's allow decision still depends on the full set of checks above.

5.5 Runtime and Offline Review

`run_runtime_loopback_scenario` drives a deterministic three-vendor workflow through SQLite receipt and authority stores with a JSON runtime admission store. It produces the treaty scope, ladder intersection, continuation, lineage, bilateral DSSE statement, local and remote receipts, and a buyer package. Producer mode stops after generation and schema validation. Full mode also invokes the buyer CLI and performs semantic package review. The offline `verify_package` path checks the linked artifacts, hashes, and signatures without access to the vendor processes.

`verify_package` calls `verify_chio_bilateral_invocation`, the strict operational verifier, and that is where the remaining receiver-state comparisons run: peer pins and key fingerprints, the strict envelope verification, ladder-manifest freshness for both participants, the revocation epoch, the resolved receipt's signature, kernel key, tool name, and hashes, the capability lease, policy-verdict agreement, and required governance evidence. Its failures carry `VerifierError::code` values such as `peer.unpinned_or_keyid_mismatch`, `peer.revoked_at_epoch`, `ladder.manifest_stale`, `capability.lease_expired_or_unknown`, and `governance.receipt_required_missing`. The pre-dispatch hook does not call this verifier.

The negative matrix reaches five surfaces. Five cases (04, 15, 16, 17, and 20) run through `ChioRuntimeAdmissionHook::evaluate`; three (05, 06, and 07) through the strict operational verifier `verify_chio_bilateral_invocation`; six (08 to 12 and 18) through the envelope verifier or signer, of which only 09 reaches `verify_chio_bilateral_dsse_envelope` while the others exercise the signature-slice envelope verifier or the signer; three (13, 14, and 19) through the partial verifier `verify_bilateral_cosign_invocation`, which its own documentation scopes as not fully conformant; and three (01, 02, and 03) through pure runtime-core validators, `bounded_treaty_receipt_view_from_verified_artifacts`

and `evaluate_cross_boundary_admission`, without any DSSE verifier. A separate dispatch-capable benchmark installs a test tool server and checks that its invocation counter remains zero after a denial and equals the iteration count after an admission. Table 1 lists the surfaces the evaluation exercises.

The supplementary manifest records the exact source files, test commands, benchmark outputs, theorem declarations, and toolchains used for this paper. It identifies each surface by file and symbol and verifies the content hashes when the package is rebuilt.

6 EVALUATION

We ask four questions. RQ1 tests whether the implementation rejects the attacks in the threat model before dispatch. RQ2 compares the independent Rust reference interpreter with the runtime bounded evaluator. RQ3 measures the pre-dispatch admission path for admitted and denied calls, the complete local buyer workflow, and their main components. RQ4 reports replay coverage and package size.

6.1 Method

Experiments ran on one host: a 12-core Arm Neoverse-N1 workstation with 46.9 GiB of memory running Linux 6.17.0-1020-oracle aarch64, with rustc 1.94.1 as pinned by `rust-toolchain.toml`. The benchmark script records this identity beside the result files. The one-minute load average at benchmark start was 3.93. All measurements use the release profile.

Three kinds of sample appear below. Complete workflows are timed as 100 runs per path after 2 warmups; each run starts release binaries, uses SQLite receipt and authority stores with a JSON runtime admission store on one host, and includes process startup, package generation, CLI execution, schema validation, and semantic review. The two pre-dispatch paths and the receipt append are timed per invocation inside the Criterion harness, so their percentiles are taken over every timed invocation and their sample counts are reported per row. The remaining components are Criterion batch means over 100 samples; for those we report the median, the mean with its 95 percent confidence interval, and the largest batch mean, never a p99. We report p99 only where it is a true per-invocation or per-run percentile over at least 100 samples.

`run-bilateral-admission.sh` retains raw per-run and per-invocation CSVs, Criterion estimates, machine and toolchain metadata, a JSON summary, and the generated TEX macros. `run-replay-corpus.sh` reports the generic capability corpus and the bilateral negative matrix separately.

6.2 RQ1: Do Attacks Deny Before Dispatch?

The corpus contains 20 cases, `PS-TH-01` through `PS-TH-20`. Each entry names the test target, expected failure code, enforcement phase, and declared dispatch state. The runner rejects duplicate identifiers, empty selections, unexpected codes, and any case that expects dispatch.

All 20 cases in Table 2 returned the expected denial code in both matrix runs. The cases reach five surfaces: five run through `ChioRuntimeAdmissionHook::evaluate` (04, 15, 16, 17, 20), three through the strict operational verifier `verify_chio_bilateral_invocation` (05, 06, 07), six through the envelope verifier or signer (08 to 12, 18), three through the partial verifier `verify_bilateral_cosign_invocation` (13, 14, 19), and three through pure runtime-core validators (01, 02, 03). None of them installs a tool server. The dispatch-capable benchmark supplies the separate non-invocation observation: its counter remained zero on every denial. Running the complete release matrix took 14.3 s, most of it in one cargo invocation per case.

One assumption cannot be tested by this corpus. If one actor controls both private keys that the receiver authorized for the two participants, the cryptographic profile still passes. We record this as `PS-A-01` rather than counting it as an attack rejected by the implementation.

6.3 RQ2: Do the Rust Evaluators Agree?

The test suite covers every atomic predicate form, then runs 1,024 generated cases for each of three properties: compound predicate evaluation, constitution admission, and finite-domain refinement. For every tested input, the independent Rust interpreter and `chio-runtime-core` returned the same result. The generator keeps its predicates within the runtime's size, depth, and node limits.

The Lean model in Section 4 defines the same data and Boolean operations and proves the finite-domain theorem. The differential suite, however, compares Rust with Rust. It is useful for finding implementation drift, not a proof that Rust is equivalent to Lean or that the surrounding admission hook is verified.

6.4 RQ3: What Does the Evaluated Path Cost?

Table 3 reports the results. The headline enforcement cost is the pre-dispatch admission path for an admitted call: 13.863 ms p50, 26.176 ms p99, and a mean of 16.081 ms (95 percent CI 14.715 to 18.010 ms) over 200 invocations. The timed body traverses the kernel's capability and federation checks, resolves the stored treaty, ladder, continuation, lineage, invocation, and bilateral statement, checks every treaty binding, verifies the envelope's two signatures, consumes the continuation, dispatches to the counting tool server, and writes the signed receipt to SQLite. The counter equalled the iteration count.

The denial beside it measured 7.073 ms p50, 18.646 ms p99, and a mean of 8.749 ms (95 percent CI 8.061 to 9.620 ms) over 400 invocations, with the counter at zero. Its fixture signs a unanimous-deny policy summary, and the hook returns `chio_treaty_policy_denied` at the policy-summary check, before the DSSE treaty-binding-reference comparisons and before signature verification. The number therefore measures the cost of refusing a unanimous denial and writing the signed SQLite denial receipt, not the full verification path, and it is a lower bound on the hook work an admitted call performs. In both fixtures the runtime admission store is in memory with a fixed clock; only the receipt store is SQLite.

Under sustained load, the same allow path admitted 3543 calls in 60 s (59.0 calls per second) with 3543 dispatches and 0 denials, and the receipt store grew by 41039.3 KiB.

The complete local buyer workflow measured 2.089 s p50 and 2.142 s p99 over 100 runs, with a mean of 2092.307 ms and a standard deviation of 25.740 ms. This number is not receiver-hook latency. It includes three-vendor process orchestration, package generation,

Table 1: Implementation surfaces exercised by the evaluation.

Surface	Purpose	Evaluation
`ChioRuntimeAdmissionHook::evaluate`	Receiver-owned resolution, treaty binding checks, and pre-dispatch decision	Runtime tests and dispatch-capable admission benchmarks
`verify_chio_bilateral_dsse_envelope`	Strict envelope verification: canonical bytes, two distinct configured keys, both signatures	Hook and kernel binding paths, federation tests, negative matrix
`verify_chio_bilateral_invocation`	Offline strict verification with peer pins, manifest freshness, revocation epoch, and receipt store	Buyer-package verification, federation tests, negative matrix
`VerifiedFederationTreatyMaterial`	Couples accepted treaty evidence to receipt construction	Kernel federation and receipt tests
`CrossKernelContinuation`	Binds parent receipt, nonce, action, target, and lifetime	Freshness and replay cases
`bounded_treaty_receipt_view_from_verified_artifacts`	Builds the bounded evaluator input from authenticated artifacts	Constructor tests and differential suite
`run_runtime_loopback_scenario`	Produces the complete local buyer package	End-to-end workflow samples
`verify_package`	Rechecks the emitted package offline	Buyer-review tests and component benchmark

Table 2: Bilateral negative matrix. All 20 cases passed.

Attack family	Cases	Rejected evidence
Binding substitution	01, 03, 05–07	Treaty, intersection, receipt, request, outcome
Signature and syntax	08–12, 18	Missing or duplicate signature, reused key, predicate, canonical form, schema
Freshness and continuity	02, 13, 15	Treaty, lease, continuation replay
Receiver trust and evidence	04, 14, 16, 17	Lineage, governance, smuggled trust
Policy outcome	19, 20	Disagreement and unanimous deny

SQLite receipt and authority stores with a JSON admission store, buyer CLI startup, schema validation, and semantic review. Producer mode measured 1820.939 ms p50 with a standard deviation of 40.038 ms. The two workflows differ in CLI and schema work as well as in buyer review, so their difference is not an ablation. The isolated offline package verifier measured 49703.524 μs median.

Among the components, receipt signing measured 229.340 μs median, strict bilateral DSSE verification 339.127 μs median, and the per-invocation SQLite append 4681.643 μs p50. The cross-boundary row is the pure treaty-scope and intersection evaluation over pre-verified evidence references; it excludes store lookups, signature verification, and the hook's binding checks. These measurements isolate real library functions, but their sum is not the complete workflow because it omits orchestration and validation outside the timed component bodies.

6.5 RQ4: Replay and Package Size

The buyer package was 51,843 bytes (50.6 KiB), identical across every workflow run because the scenario is deterministic. The replay script validated 50 generic capability fixtures in 1 s and, separately, 20 bilateral negative cases in 14 s. The generic corpus checks stable capability verdicts. The bilateral corpus checks the failures in Table 2; its coverage is not inflated by the larger generic count.

6.6 Measurement Limits

The deterministic single-host setup removes network variance and makes package comparison reproducible. It does not estimate wide-area latency, concurrent load, or partial-failure behavior. The sample counts characterize this machine and are not service-level objectives. The supplementary package records the source, scripts, raw samples, and environment needed to reproduce the experiment.

7 DISCUSSION

Why the receiver owns resolution. A signature authenticates bytes, not the authority used to interpret them. If the sender can also supply the peer keys, revocation state, treaty manifest, or policy bundle, a valid signature can authenticate a self-appointed trust domain. Receiver-owned resolution turns those references into lookups against local state. Bilateral signing establishes agreement on one statement, while each participant keeps control of its own admission decision.

Two keys and two operators. The protocol establishes possession of two distinct authorized private keys over identical bytes. It cannot tell whether separate organizations controlled those keys or evaluated policy independently. A deployment that needs that stronger property must add identity governance, separated key custody, or attested execution.

Enforcement before evidence review. The buyer package is useful because it preserves the same bindings enforced at dispatch. It is evidence of what the configured kernels and keys accepted, not a substitute for pre-dispatch checks. Offline verification can detect an internally inconsistent package, but it cannot recover a compromised receiver or prove how a remote process behaved.

Local technical authority. Receiver-owned admission gives an organization final technical control over whether a foreign agent call crosses its own tool boundary. When this property is described as programmable sovereignty, the term refers only to that local control. It carries no legal, territorial, or institutional status, and every counterparty remains free to apply a different policy.

8 RELATED WORK

Capabilities and agent isolation. KeyKOS, EROS, object-capability systems, Capsicum, and seL4 established explicit, attenuable authority and small enforcement boundaries [7, 9, 12, 13, 17, 22]. Chio applies that discipline to agent tool calls and retains a signed artifact of the admission decision. IsolateGPT separates agent applications behind information-flow gates, while SAGA distributes user-controlled authorization tokens [20, 23]. Recent analyses identify confused-deputy and authority-boundary failures as central risks in agentic systems [2]. Receiver-owned bilateral admission is complementary: it governs the call after authority crosses an organizational boundary.

Authenticated evidence. DSSE authenticates a typed payload without making its internal policy semantics universal [16]. in-toto

Table 3: Release-profile results on the host described in the Method. Rows above the "Criterion batch means" sub-heading are per-invocation or per-run distributions with true percentiles; the rows under it are Criterion batch means, for which the largest batch mean replaces a percentile; the final row summarizes the sustained-load run. CI is the 95 percent confidence interval of the mean.

Measured path	p50	p99	mean [95% CI]	*n*	Included work
Pre-dispatch admission, allow	13.863 ms	26.176 ms	16.081 [14.715, 18.010] ms	200	Kernel checks, in-memory admission store, treaty hook with binding checks and envelope verification, dispatch to a counting tool server, signed SQLite receipt
Pre-dispatch admission, denial	7.073 ms	18.646 ms	8.749 [8.061, 9.620] ms	400	Kernel checks, in-memory admission store, treaty hook up to the policy-summary check, signed SQLite denial receipt
SQLite receipt append	4681.643 μs	13566.644 μs	5554.817 [5365.278, 5752.154] μs	500	Persistent receipt store; signing excluded
Complete local buyer workflow	2088.886 ms	2141.921 ms	2092.307 [2087.402, 2097.369] ms	100	Three-vendor producer, SQLite receipt and authority stores, JSON admission store, buyer CLI, schema and semantic review
Producer workflow	1820.939 ms	2033.854 ms	1824.924 [1817.759, 1833.514] ms	100	Three-vendor producer, SQLite receipt and authority stores, JSON admission store, package generation and schema validation
Criterion batch means	median	max	mean [95% CI]	batches	
Receipt sign	229.340 μs	270.205 μs	229.932 [229.377, 230.888] μs	100	Ed25519 over the evaluated receipt shape
Receipt verify	337.470 μs	344.582 μs	337.586 [337.225, 337.955] μs	100	Ed25519 over the evaluated receipt shape
Strict bilateral DSSE verify	339.127 μs	350.904 μs	339.291 [339.032, 339.618] μs	100	Two signatures and embedded treaty bindings
Cross-boundary admission allow	20.688 μs	21.207 μs	20.701 [20.687, 20.716] μs	100	Pure treaty-scope and intersection evaluation over pre-verified evidence references; no store, signature, or hook work
Offline buyer-package verify	49703.524 μs	51598.253 μs	49724.725 [49696.672, 49769.631] μs	100	Complete three-vendor package
Sustained admitted load	3543 admitted calls in 60 s (59.0 calls/s); 3543 dispatches; 0 denials; receipt store grew 41039.3 KiB				The allow path repeated back to back for the wall-clock window; store growth measured on disk

Table 4: Assumptions outside the evaluated receiver. Identifiers refer to the repository's assumption registry, `formal/assumptions.toml`.

Assumption	Required for	Failure consequence
Ed25519 and SHA-256 behave as specified, and DSSE binds payload and type (`ASSUME-ED25519`, `ASSUME-SHA256`)	Signature, digest, and envelope integrity	Forgery or ambiguous identity invalidates the bindings
Canonical JSON matches RFC 8785 on Unicode strings, bounded integers, arrays, and UTF-16-ordered objects (`ASSUME-CANONICAL-JSON`); float rendering outside that domain is assumed stable, and signed payloads carry integers above $2^{53} - 1$ exactly rather than as doubles	Stable statement and receipt bytes [15]	Equivalent data can acquire different identities
The injected clock reports operator-accepted Unix time within the deployment tolerance (`ASSUME-OS-CLOCK`)	Continuation, lease, and treaty validity intervals	Stale material admits, or fresh material denies
SQLite commits each single-row write atomically (`ASSUME-SQLITE-ATOMICITY`); cross-row crash recovery is not assumed	Receipt append and single-use continuation consumption	A torn write can lose a receipt or permit a replay
Tool-server subprocess and OS process boundaries hold (`ASSUME-SUBPROCESS-ISOLATION`)	Non-dispatch of denied calls; the invocation counter observes only kernel-mediated calls	A tool reachable outside the kernel bypasses admission
The receiving edge sets the federated origin class, and the caller cannot (`ASSUME-FEDERATED-ORIGIN-CLASSIFICATION`)	Routing cross-organization requests into the treaty hook	A misclassified request follows the local path and bypasses treaty admission
Receiver kernel and stores are trustworthy	Correct resolution and continuation state	A compromised receiver can admit or misrecord calls
Authorized keys have the intended organizational custody (`PS-A-01`)	Interpreting two signatures as bilateral control	One actor with two keys satisfies the cryptographic checks

and SLSA use signed statements to describe software supply-chain provenance [19, 21]; transparency logs make authenticated statements auditable over time [10, 18]. Our bilateral statement instead binds an invocation and the receiver's admission evidence. It can cite supply-chain provenance, but provenance alone does not establish a live treaty, continuation, policy verdict, or receipt binding.

Verified authorization. Cedar combines a formal policy semantics with a production Rust evaluator and differential testing [3, 5]. Large verified systems such as seL4, IronFleet, and Project Everest connect stronger implementation claims to explicit refinement arguments [8, 9, 14]. Our Lean 4 development [4] specifies a bounded receipt-predicate syntax and proves its finite-domain refinement check. The Rust evidence is a differential test between independent evaluators, not a refinement proof for the runtime.

Cross-domain agreement. PBFT and federated Byzantine agreement establish replicated decisions under quorum assumptions [1, 11]. IBC verifies commitments across consensus zones [6]. Bilateral admission solves a receiver-side authorization problem rather than replicated consensus. It lets one receiver decide whether a co-signed statement is sufficient for one local dispatch. It neither converges histories nor makes a global decision.

9 LIMITATIONS

Table 4 lists the assumptions the evaluated receiver depends on. The Lean theorem applies to the receipt fields, predicate forms, and finite domain defined in Section 4. It proves the checker correct for that supplied domain. Completeness of the domain, parsing, canonicalization, cryptography, clocks, stores, complexity-limit enforcement, and the receiver hook remain outside the theorem. The differential corpus stays within those limits and cannot establish universal implementation equivalence.

Two valid signatures establish control of two configured keys. Organizational independence, honest policy evaluation, uncompromised key custody, and execution inside an attested kernel require additional deployment controls. HSM-, KMS-, or TEE-backed keys could strengthen key custody, but we did not evaluate those configurations.

The performance results come from a deterministic single-host setup with SQLite receipt stores and in-memory or JSON admission stores. The experiments exclude wide-area transport, mTLS setup, partitions, concurrent load, large receipt stores, hardware security modules, trusted execution environments, and crash recovery during continuation consumption. The sample counts characterize one host and do not support operational tail-latency targets.

The 20 negative cases cover named attacks, not an unbounded adversary. Six of them stop in the envelope verifier or signer and three in pure validators, before a dispatch-capable runtime surface exists. The dispatch-capable benchmark supplies the signed SQLite receipt and zero-invocation observation for a unanimous policy denial and the admitted-path cost for an allow; neither should be generalized to every internal error in Chio.

Finally, the buyer package establishes internal consistency under its configured keys. It cannot prove remote process integrity, prevent an authorized kernel from lying, or compel another organization to accept the same evidence.

10 CONCLUSION

A vendor signature is not enough to authorize a cross-organization agent call. The receiver must decide which treaty, keys, request, continuation, policy result, and prior receipts give that signature meaning. Chio performs this check inside the receiving kernel before tool dispatch and carries the accepted bindings into the signed receipt.

In the evaluated configuration, all 20 named attacks denied. The pre-dispatch admission path for an admitted call measured 13.863 ms p50; a unanimous policy denial measured 7.073 ms p50 and left the tool invocation counter at zero. The separate local buyer workflow generated, stored, and verified the complete three-vendor package in 2.089 s p50. Lean proves the exact finite-domain condition for its checker, and the independent Rust interpreter matched the runtime evaluator throughout the generated corpus. Together, these results show that receiver-owned bilateral admission is practical as an enforcement step, while keeping its trust and formal boundaries explicit.

REFERENCES

[1] Miguel Castro and Barbara Liskov. 1999. Practical Byzantine Fault Tolerance. In *Proceedings of the Third Symposium on Operating Systems Design and Implementation*. USENIX Association, Berkeley, CA, USA, 173–186. https://www.usenix.org/conference/osdi-99/presentation/practical-byzantine-fault-tolerance

[2] Mihai Christodorescu, Earlence Fernandes, Ashish Hooda, Somesh Jha, Johann Rehberger, Kamalika Chaudhuri, Xiaohan Fu, Khawaja Shams, Guy Amir, Jihye Choi, Sarthak Choudhary, Nils Palumbo, Andrey Labunets, and Nishit V. Pandya. 2025. Systems Security Foundations for Agentic Computing. arXiv:2512.01295. https://arxiv.org/abs/2512.01295

[3] Joseph W. Cutler, Craig Disselkoen, Aaron Eline, Shaobo He, Kyle Headley, Michael Hicks, Kesha Hietala, Eleftherios Ioannidis, John Kastner, Anwar Mamat, Darin McAdams, Matt McCutchen, Neha Rungta, Emina Torlak, and Andrew Wells. 2024. Cedar: A New Language for Expressive, Fast, Safe, and Analyzable Authorization. *Proceedings of the ACM on Programming Languages* 8, OOPSLA1 (2024), 123:1–123:30. https://doi.org/10.1145/3649835

[4] Leonardo de Moura and Sebastian Ullrich. 2021. The Lean 4 Theorem Prover and Programming Language. In *Automated Deduction – CADE 28*. Springer, Cham, Switzerland, 625–635. https://doi.org/10.1007/978-3-030-79876-5_37

[5] Craig Disselkoen, Aaron Eline, Shaobo He, Kyle Headley, Michael Hicks, Kesha Hietala, John Kastner, Anwar Mamat, Matt McCutchen, Neha Rungta, Bhakti Shah, Emina Torlak, and Andrew Wells. 2024. How We Built Cedar: A Verification-Guided Approach. arXiv:2407.01688. https://arxiv.org/abs/2407.01688

[6] Christopher Goes. 2020. The Interblockchain Communication Protocol: An overview. arXiv:2006.15918. https://arxiv.org/abs/2006.15918

[7] Norman Hardy. 1985. KeyKOS Architecture. *ACM SIGOPS Operating Systems Review* 19, 4 (1985), 8–25. https://doi.org/10.1145/858336.858337

[8] Chris Hawblitzel, Jon Howell, Manos Kapritsos, Jacob R. Lorch, Bryan Parno, Michael L. Roberts, Srinath Setty, and Brian Zill. 2015. IronFleet: Proving Practical Distributed Systems Correct. In *Proceedings of the 25th Symposium on Operating Systems Principles*. ACM, New York, NY, USA, 1–17. https://doi.org/10.1145/2815400.2815428

[9] Gerwin Klein, Kevin Elphinstone, Gernot Heiser, June Andronick, David Cock, Philip Derrin, Dhammika Elkaduwe, Kai Engelhardt, Rafal Kolanski, Michael Norrish, Thomas Sewell, Harvey Tuch, and Simon Winwood. 2009. seL4: Formal Verification of an OS Kernel. In *Proceedings of the 22nd ACM Symposium on Operating Systems Principles*. ACM, New York, NY, USA, 207–220. https://doi.org/10.1145/1629575.1629596

[10] Ben Laurie, Emilia Messeri, and Rob Stradling. 2021. Certificate Transparency Version 2.0. RFC 9162. https://doi.org/10.17487/RFC9162

[11] David Mazieres. 2015. *The Stellar Consensus Protocol: A Federated Model for Internet-level Consensus*. Technical Report. Stellar Development Foundation, San Francisco, CA, USA. https://stellar.org/papers/stellar-consensus-protocol.pdf

[12] Mark S. Miller. 2006. *Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control*. Ph. D. Dissertation. Johns Hopkins University. https://jscholarship.library.jhu.edu/bitstream/handle/1774.2/873/markm-thesis.pdf

[13] Toby Murray. 2010. *Analysing the Security Properties of Object-Capability Patterns*. Ph. D. Dissertation. University of Oxford. https://www.cs.ox.ac.uk/files/3080/thesis-FINAL-15-07-10.pdf

[14] Project Everest. 2017. Project Everest: Verified Secure Implementations for the HTTPS Ecosystem. https://project-everest.github.io/

[15] Anders Rundgren, Bret Jordan, and Samuel Erdtman. 2020. JSON Canonicalization Scheme (JCS). RFC 8785. https://doi.org/10.17487/RFC8785

[16] Secure Systems Lab. 2024. DSSE: Dead Simple Signing Envelope (v1.0.2). https://github.com/secure-systems-lab/dsse

[17] Jonathan S. Shapiro, Jonathan M. Smith, and David J. Farber. 1999. EROS: A Fast Capability System. In *Proceedings of the Seventeenth ACM Symposium on Operating Systems Principles*. ACM, New York, NY, USA, 170–185. https://doi.org/10.1145/319151.319163

[18] Sigstore Project. 2021. Sigstore Security Model. https://docs.sigstore.dev/about/security/

[19] SLSA Framework. 2023. Supply-chain Levels for Software Artifacts Specification (v1.0). https://slsa.dev/spec/latest/

[20] Georgios Syros, Anshuman Suri, Jacob Ginesin, Cristina Nita-Rotaru, and Alina Oprea. 2026. SAGA: A Security Architecture for Governing AI Agentic Systems. In *33rd Network and Distributed System Security Symposium (NDSS)*. Internet Society, Reston, VA, USA, 1–19. https://www.ndss-symposium.org/ndss-paper/saga-a-security-architecture-for-governing-ai-agentic-systems/

[21] Santiago Torres-Arias, Hammad Afzali, Trishank Karthik Kuppusamy, Reza Curtmola, and Justin Cappos. 2019. in-toto: Providing Farm-to-Table Guarantees for Bits and Bytes. In *28th USENIX Security Symposium*. USENIX Association, Berkeley, CA, USA, 1393–1410. https://www.usenix.org/conference/usenixsecurity19/presentation/torres-arias

[22] Robert N. M. Watson, Jonathan Anderson, Ben Laurie, and Kris Kennaway. 2010. Capsicum: Practical Capabilities for UNIX. In *19th USENIX Security Symposium*. USENIX Association, Berkeley, CA, USA, 29–46. https://www.usenix.org/conference/usenixsecurity10/capsicum-practical-capabilities-unix

[23] Yuhao Wu, Franziska Roesner, Tadayoshi Kohno, Ning Zhang, and Umar Iqbal. 2025. IsolateGPT: An Execution Isolation Architecture for LLM-Based Agentic Systems. In *32nd Network and Distributed System Security Symposium (NDSS)*. Internet Society, Reston, VA, USA, 1–18. https://www.ndss-symposium.org/ndss-paper/isolategpt-an-execution-isolation-architecture-for-llm-based-agentic-systems/